

Documento maestro por fases — Frontend (Web + Desktop + Landing)

Este documento es la plantilla universal para construir el **frontend** de un negocio tipo tienda (electrónica/Arduino) con visión “de industria”, pero enfocado en **UX/UI real** (no solo “hacer pantallas”).

Incluye:

- App interna (operación): **Web (React/Next.js)** y/o **Desktop (JavaFX)**
- Web propagandística: **Landing Page** (marketing)
- Fases con: **Objetivo + Pasos + Precauciones + Observaciones + Salidas**
- Mini-ingeniería de diseño (IA, user flows, wireframes, mockups)
- Arquitectura frontend (carpetas, módulos, estado, validación, performance)
- Criterios para elegir stack y hosting

Contexto del negocio: tienda con operación real (mostrador + pedidos WhatsApp + inventario + reportes). Escenario moderado: ~**5000 productos (SKUs)**.

Glosario mínimo

- **Landing page**: web propagandística para atraer clientes (no es el sistema interno).
 - **Wireframe**: maqueta simple (sin diseño final) para validar estructura.
 - **Mockup**: maqueta con diseño visual (tipografía, colores, spacing).
 - **Prototipo**: wireframe/mockup interactivo (clickeable).
 - **User flow**: ruta típica de pantallas (ej: vender → cobrar → cerrar).
 - **IA (Information Architecture)**: mapa de navegación (menús, secciones, jerarquía).
 - **UI Kit / Design System**: componentes consistentes (botones, tablas, inputs, etc.).
 - **POS (Point of Sale)**: pantalla de caja/venta rápida.
 - **RBAC**: permisos por roles (admin, vendedor, bodega).
 - **SKU**: “unidad vendible” identificable (código único por variante: marca, voltaje, versión, etc.).
-

Reglas globales (frontend bien hecho)

Precauciones globales

- **Tu UI debe evitar errores humanos**, no solo “dejar hacer cosas”.
- **No dependas de memoria del usuario**: todo debe estar en la pantalla (stock, estado, total, permisos).
- **No hagas pantallas gigantes sin orden**: divide en secciones y usa tabs/pasos.
- **No rompas el flujo de caja**: vender debe ser el camino más corto del sistema.
- **Si algo es lento, el negocio lo abandona**: 5000 productos exige búsqueda, filtros y performance.

Observaciones globales

- El frontend “de tienda” es 70% UX (flujo) y 30% tecnología.

- Un panel administrativo exitoso se siente:
- rápido
- claro
- con pocos clics
- resistente a errores
- El frontend se vuelve caro cuando:
- no hay contrato API estable
- no hay diseño de navegación
- todo son formularios sin patrones

Entregables globales

- IA + user flows + wireframes
- UI kit mínimo (componentes)
- Arquitectura de carpetas
- Política de errores y validación
- Plan de deployment (web + landing)

Mapa de fases (vista rápida)

Fase -1 — Estrategia del producto (3 superficies)

1. Fase 0 — Alcance UX (qué pantallas existen en V1)
2. Fase 1 — IA + Navegación (mapa del sistema)
3. Fase 2 — User flows críticos (caja / inventario / pedidos)
4. Fase 3 — Wireframes (maquetas) por módulo
5. Fase 4 — UI Kit / Design System mínimo
6. Fase 5 — Arquitectura frontend (carpetas + módulos + estado)
7. Fase 6 — Contrato con backend (API client + auth + RBAC)
8. Fase 7 — Desarrollo por módulos (implementación)
9. Fase 8 — Testing y QA (manual + automatizado)
10. Fase 9 — Performance (5000 items) + UX hardening
11. Fase 10 — Deploy (web app + landing) + distribución desktop
12. Fase 11 — Mantenimiento + crecimiento (V2/V3)

Fase -1 — Estrategia del producto (3 superficies)

Objetivo

Separar claramente qué es cada cosa:

1. **App interna (operación):** caja, inventario, compras, ventas, pedidos.
2. **Landing page (marketing):** atraer clientes, WhatsApp, catálogo público (opcional).
3. **Desktop (si se decide):** operación en PC con “sensación de sistema cerrado”.

Pasos

1. Definir si la operación será:
2. Web (panel)
3. Desktop (JavaFX)
4. Ambos (ideal para documento maestro)
5. Definir la landing:
6. informativa
7. CTA (WhatsApp)
8. mapa/local
9. catálogo simple (opcional)

Precauciones

- No mezclar “landing” con “panel interno”: son productos distintos.
- El panel necesita RBAC y seguridad; la landing debe ser rápida y pública.

Observaciones

- Estrategia típica para crecer:
- Arrancas con **desktop o web interna simple**
- Luego pones **landing** para presencia pública
- Después integras catálogo público si conviene

Salidas

- Lista de 3 repos o 3 apps separadas (recomendado):
 - `store-admin-web/`
 - `store-desktop/` (opcional)
 - `store-landing/`
-

Fase 0 — Alcance UX (qué pantallas existen en V1)

Objetivo

Definir pantallas reales que el negocio sí usa.

Pantallas mínimas V1 (App interna)

1. Login
2. Dashboard simple (opcional)
3. POS (Venta rápida)
4. Productos (listado + editar)
5. Inventario (stock + movimientos)
6. Compras (entrada de inventario)

7. Pedidos WhatsApp (pipeline)
8. Clientes
9. Proveedores
10. Reportes (PDF / export)
11. Usuarios/Roles (admin)

Precauciones

- Si no existe POS, el sistema se vuelve “lento” y no se usa.
- No metas reportes avanzados hasta que ventas/inventario estén sólidos.

Observaciones

- La V1 de frontend debe ser:
- usable en 1 semana de entrenamiento
- sin pantallas raras ni pasos redundantes

Salidas

- Lista oficial de pantallas V1
 - Lista de pantallas V2
-

Fase 1 — IA + Navegación (mapa del sistema)

Objetivo

Que el usuario nunca se pierda: menú y rutas coherentes.

IA recomendada (panel)

- Inicio
- Caja (POS)
- Pedidos
- Catálogo
- Categorías
- Productos
- SKUs
- Inventario
- Stock
- Movimientos
- Ajustes
- Compras
- Ventas
- Clientes
- Proveedores
- Reportes
- Administración
- Usuarios

- Roles
- Config

Precauciones

- No hagas 20 items en el menú sin jerarquía.
- El usuario de tienda necesita entrar con 1 clic a **Caja**.

Observaciones

- La IA es “arquitectura de interacción”: ahorra bugs y tiempo.

Salidas

- Mapa de navegación + rutas
-

Fase 2 — User flows críticos

Objetivo

Definir los caminos que más dinero/errores generan.

Flujos críticos (prioridad)

1) Venta rápida (POS)

Buscar producto → agregar → ajustar cantidades → aplicar descuento (si existe) → cobrar → cerrar

2) Compra (entrada)

Seleccionar proveedor → agregar ítems → confirmar recepción → inventario sube

3) Pedido WhatsApp

Crear pedido → confirmar → reservar (lógico) → empacar → entregar/enviar → (opcional) convertir a venta

4) Devolución

Buscar venta → seleccionar ítems → motivo → confirmar → inventario ajusta

Precauciones

- Si un flujo tiene más de 6–8 acciones, se vuelve lento y el usuario se equivoca.

Observaciones

- Diseño UX = reducir pasos y prevenir errores.

Salidas

- Diagrama de flujos (simple, por texto o herramienta)
-

Fase 3 — Wireframes (maquetas) por módulo

Objetivo

Definir layout antes de diseñar bonito.

Plantillas GUI típicas (lo que estabas recordando)

Estas son las plantillas base que se repiten en apps administrativas.

Plantilla A: Listado con filtros (Admin Table)

- Barra superior: título + acciones
- Filtros: search, estado, rango de fecha
- Tabla: columnas + acciones por fila
- Paginación

Plantilla B: Formulario de edición (Create/Edit)

- Secciones
- Validación visible
- Botones: Guardar / Cancelar

Plantilla C: Detalle + ítems (Master/Detail)

- Encabezado: estado, totales
- Sección de ítems (tabla editable)
- Panel lateral: pagos/envío/notas

Plantilla D: Wizard / pasos (para procesos)

- Paso 1: datos base
- Paso 2: ítems
- Paso 3: confirmación

Plantilla E: Panel de estado (Pipeline)

- Columnas por estado (pendiente, confirmado, empacado...)
 - Tarjetas con acciones rápidas
-

Wireframes mínimos (texto)

1) POS — Venta rápida

```
[ POS | Caja ] [Usuario: Vendedor]
-----
Buscar SKU / Nombre: [_____] [Agregar]

Carrito:
| SKU | Producto | Cant | Precio | Subtotal | (-) (+) |
|-----|-----|-----|-----|-----|-----|

[Notas] [Descuento] Total: $____
Pago: ( ) Efectivo ( ) Transferencia Monto: [____]

[ CERRAR VENTA ] [ GUARDAR BORRADOR ]
```

2) Productos — Listado

```
[Productos]
Search: [____] Categoria: [____] Activo: [Si/No]

| SKU | Nombre | Categoria | Stock | Precio | Estado | Acciones |

[<<] [<] Page 1 of N [>] [>>]
```

3) Pedido WhatsApp — Pipeline

```
[Pedidos]
Pendiente | Confirmado | Empacado | Enviado | Entregado
[ cards ] [ cards ] [ cards ] [ cards ] [ cards ]

Acciones por card: Ver / Confirmar / Empacar / Enviar
```

4) Inventario — Stock + Movimientos

```
[Inventario]
Search: [____] Bajo stock: [x]

Stock:
| SKU | Producto | Disponible | Reservado | Min | Estado |

Movimientos (tab):
| Fecha | Tipo | SKU | Cant | Motivo | Usuario |
```

Precauciones

- Wireframe primero, diseño después.
- Validar con el cliente/usuario real antes de gastar tiempo en UI final.

Observaciones

- Un wireframe bueno reduce refactor 10×.

Salidas

- Wireframes por pantalla (mínimo V1)
-

Fase 4 — UI Kit / Design System mínimo

Objetivo

Consistencia visual + velocidad de desarrollo.

Componentes mínimos

- Botón (primario/secundario/peligro)
- Input + label + error
- Select
- Tabla
- Modal
- Toast/alert
- Badge de estado
- Tabs
- Breadcrumb (opcional)

Precauciones

- Si cada pantalla tiene estilos distintos, el proyecto se vuelve lento.

Observaciones

- Admin panel bonito ≠ admin panel usable. La usabilidad manda.

Salidas

- UI kit base
-

Fase 5 — Arquitectura frontend (carpetas + módulos + estado)

Objetivo

Estructura escalable (sin monstruo).

Arquitectura sugerida (React/Next.js)

- `app/` o `pages/` (rutas)
- `modules/` (por dominio)
- `shared/` (UI kit, utils)
- `services/` (API client)
- `stores/` (estado global si aplica)

Estado (reglas prácticas)

- Estado local: inputs/formularios
- Estado de servidor: listas, fetches (ideal con React Query)
- Estado global mínimo: auth, usuario, permisos

Precauciones

- No metas todo en Redux por costumbre.
- La mayor fuente de bugs es el estado duplicado.

Observaciones

- Con 5000 productos, necesitas:
- búsqueda eficiente
- paginación
- debounced search

Salidas

- Árbol de carpetas + reglas de imports

Fase 6 — Contrato con backend (API client + auth + RBAC)

Objetivo

Conectarse sin caos y sin fugas de seguridad.

Pasos

1. Cliente HTTP (fetch/axios) con base URL
2. Interceptor de errores
3. Auth JWT:
4. guardar token (preferible en memoria o storage con cuidado)
5. refrescar sesión (si se implementa)
6. RBAC en UI:
7. ocultar botones
8. bloquear rutas
9. Manejo de errores "humanos":
10. stock insuficiente
11. no permitido
12. venta cerrada

Precauciones

- No confiar solo en UI: backend manda.
- Mensajes de error deben ser claros para el usuario de tienda.

Observaciones

- Si Swagger está bien, el frontend se implementa rapidísimo.

Salidas

- API client listo
- Guard de rutas

Fase 7 — Desarrollo por módulos

Objetivo

Construir pantallas reales en orden.

Orden recomendado (V1)

1. Auth + layout base
2. POS
3. Productos (listado + create/edit)
4. Inventario (stock + movimientos)
5. Compras
6. Pedidos WhatsApp
7. Ventas (historial)
8. Reportes PDF
9. Usuarios/Roles

Precauciones

- Si el POS no queda sólido, el sistema pierde valor.

Observaciones

- El frontend es repetitivo: tablas + formularios. Aquí el UI kit te salva.

Salidas

- Pantallas completadas por módulo
-

Fase 8 — Testing y QA

Objetivo

Evitar errores operativos.

QA Manual (checklist)

- Vender rápido sin lag
- No permitir cantidades negativas
- Bloquear cierre sin items
- Mostrar stock actualizado
- Permisos correctos por rol

Automatización (cuando convenga)

- Tests de componentes críticos
- Smoke tests (login + POS)

Precauciones

- Testear todo es caro. Testea lo que produce dinero y evita pérdidas.

Observaciones

- Un test E2E mínimo te da confianza para refactor.

Salidas

- QA checklist + tests clave
-

Fase 9 — Performance (5000 productos) + UX hardening

Objetivo

Que el panel no sea lento.

Técnicas esenciales

- Paginación server-side
- Búsqueda con debounce
- Evitar renders gigantes
- Tablas virtualizadas (si hace falta)
- Cache de consultas

Precauciones

- Cargar 5000 filas en una tabla de golpe = muerte.

Observaciones

- La performance es parte del UX.

Salidas

- Panel rápido
-

Fase 10 — Deploy (web app + landing) + Desktop

Objetivo

Publicar y operar.

Opciones de deploy (visión general)

Web App Admin

- **Vercel (Next.js)**: rápido, ideal para web moderna
- **Netlify (React)**: simple para SPA
- **PaaS (Railway/Render)**: full stack fácil
- **VPS**: control total

Landing page

- Ideal en Vercel/Netlify

- SEO + WhatsApp CTA

Desktop JavaFX

- Distribución: instalador/zip
- Actualizaciones: manual o sistema de update

Precauciones

- Si el cliente no entiende deploy, simplifica (PaaS).
- VPS = tú eres DevOps.

Observaciones

- Estrategia típica freelance:
- Landing en Vercel
- Admin web en PaaS o VPS
- Desktop solo si el cliente lo pide

Salidas

- Sistema operable
-

Fase 11 — Mantenimiento + crecimiento (V2/V3)

Objetivo

Evolucionar sin romper la operación.

Crecimientos típicos

- Multi-bodega
- Reservas reales
- Catálogo público
- Integración WhatsApp API
- Reportes automáticos

Precauciones

- Cada feature nueva debe pasar por:
- alcance
- impacto en UX
- impacto en inventario

Observaciones

- El mejor freelance no es el que hace más features. Es el que hace **features que no rompen el negocio.**

Salidas

- Roadmap
-

Checklist de salida V1 (Frontend)

✓ Login + RBAC ✓ POS usable y rápido ✓ Catálogo CRUD funcional ✓ Inventario visible y consistente
✓ Pedidos WhatsApp operables ✓ Reportes PDF básicos ✓ UX con validación y errores humanos ✓
Deploy definido (admin + landing)